

AgentGuard: Trace-Conditioned Runtime Governance for Intent-Aligned Tool Use in AI Agents

Nandan Prince

University of Maryland, College Park
Draft research manuscript

Abstract

AI agents increasingly operate through external tools that can read files, send emails, query databases, modify cloud resources, and trigger irreversible side effects. Existing runtime guardrails primarily evaluate individual tool calls for generic safety risks such as credential exposure, shell misuse, data exfiltration, or unsafe file operations. This action-level view is necessary but insufficient: many consequential failures are not obviously unsafe in isolation. A tool call may be syntactically valid and generically safe while still being misaligned with the user’s intent, premature within the session, excessive in scope, or induced by malicious content returned from a previous tool invocation.

We propose AGENTGUARD, a trace-conditioned runtime governance framework for AI agent tool use. AGENTGUARD intercepts proposed tool calls before execution and evaluates them against the user intent, prior tool-call trajectory, argument scope, tool-output context, and a memory of previously approved and blocked traces. To support this evaluation, we introduce INTENTTRACEBENCH, a structured benchmark design for raw agent execution traces collected from live agents across benign, ambiguous, and adversarial sessions. Each trace separates raw execution data, human or judge labels, and guard-generated scores to avoid benchmark contamination. We define an experimental protocol for evaluating whether session-level traces improve detection of intent drift, scope creep, argument drift, prompt-injection-induced misuse, and premature irreversible actions compared with stateless runtime guards. This manuscript is a research-design draft: quantitative results are intentionally left as placeholders until the proposed experiments are run.

1 Introduction

AI agents are moving from passive text generation toward delegated action. They plan, call tools, inspect outputs, and continue acting across multiple steps. This creates a runtime safety problem that cannot always be reduced to a single command, API call, or function invocation. In tool-using agents, the dangerous unit is often a *trajectory*.

Consider three representative failures. First, a user asks an email agent to summarize a budget thread and draft a reply; the agent calls `gmail_send` instead of `gmail_draft`. The action is not generically malicious, but it violates the user’s requested level of autonomy. Second, a user asks a file agent to summarize `ql_report.pdf`; the agent reads the report, writes a summary, and then reads `financial_data.csv` for “additional context.” Reading a file may be safe in isolation, but the second read exceeds task scope. Third, a user asks an agent to inspect a recent email; the email contains an instruction to forward internal messages to an external address, and the agent treats this tool output as an instruction source. The proposed send action is structurally normal but semantically hijacked.

Recent work validates runtime interception as a promising architecture. AgentTrust intercepts tool calls before execution and combines shell normalization, static policies, SafeFix suggestions, RiskChain detection, and LLM-as-judge evaluation [1]. AgentDojo, ToolEmu, AgentHarm, R-Judge, AgentTrap, and AgenTRIM study agent safety, prompt injection, tool risk, and execution traces from complementary angles [2–7]. However, there remains a practical gap between action-level safety and intent-relative runtime governance: a tool call can be safe in general but wrong for *this* user goal, at *this* step, after *this* sequence of prior actions.

This paper studies the hypothesis that **session-level traces improve runtime detection of unsafe or intent-misaligned tool calls compared with stateless action-level guards, while preserving benign task completion and acceptable**

Table 1: Action-level safety and intent-relative governance answer different questions.

Scenario	Action-level question	Intent-relative question
Email send	Is <code>gmail_send</code> allowed?	Did the user ask to send or only draft?
File read	Is reading this file dangerous?	Is this file within the task scope?
External API call	Is the endpoint blocked?	Does this call follow from the prior sequence?
Tool output instruction	Is the next action malformed?	Was the action induced by untrusted tool output?

latency. We make the benchmark and ablation study central. The system is not claimed as a complete solution to agent safety. Instead, we define a concrete path to test whether trace-conditioned governance adds measurable value over existing stateless and rule-based approaches.

Contributions. This draft proposes four contributions:

1. **A session-level trace schema** that separates raw execution traces, human or judge labels, and guard outputs.
2. **INTENTTRACEBENCH**, a benchmark design for evaluating runtime tool-call governance on benign, ambiguous, and adversarial agent trajectories.
3. **AGENTGUARD**, a trace-conditioned runtime guard that evaluates proposed tool calls using user intent, prior sequence, argument scope, tool-output context, and approved/blocked trace memory.
4. **An empirical protocol** with baselines, ablations, latency measurement, and hackathon-vs-paper scope boundaries.

2 Motivation and Gap

The key failure mode is not that existing guards are useless. Static policy and action-level interception are necessary. Shell deobfuscation, credential checks, schema validation, and network exfiltration rules are essential safety floors. The missing layer is different: *intent-relative appropriateness*. A call can be allowed by generic safety rules and still be misaligned with the user request.

AgentTrust is the closest architectural foundation. It reports a runtime interception system with a 300-scenario internal benchmark and an additional 630 independently constructed scenarios [1]. The system validates the value of MCP-compatible interception. But the relevant gap for this work is methodological: action-level scenarios do not fully test whether the guard can reason over user intent, prior actions, argument drift, and session-level scope creep. Our uploaded project analysis identifies the same gap: it frames AgentTrust’s stateless evaluation as a key opening and identifies semantic intent alignment, learned trace memory, and live agent demonstration as the primary differentiators for GoldenPath Guard/AgentGuard.

The strongest research framing is therefore not “another MCP firewall.” It is:

Can runtime governance improve when the proposed tool call is evaluated against the session trajectory rather than as an isolated action?

3 Research Questions

We organize the work around five research questions.

RQ1: Does session context improve runtime detection? We test whether user intent, prior tool calls, argument history, and tool-output context improve detection over stateless action-level guards.

RQ2: Which dimensions of context matter most? We ablate user intent, prior sequence, arguments, tool-output dependency, trace memory, negative traces, and cumulative drift.

RQ3: Does trace memory improve beyond static policies and LLM judges? We evaluate whether retrieval over approved and blocked traces improves detection of scope creep, argument drift, and trajectory drift.

RQ4: Can safety improve without destroying task completion? We measure harmful execution prevention together with benign task completion and false intervention rate.

RQ5: Can the guard run within practical latency budgets? We measure policy latency, retrieval latency, LLM fallback latency, full-guard p50/p95 latency, and percentage of calls resolved without LLM fallback.

4 Problem Formulation

Let a user session be represented as

$$S = (u, a, p, T_1, T_2, \dots, T_n), \quad (1)$$

where u is the user intent, a is the agent configuration, p is the system/policy context, and T_i is the i -th proposed or executed tool call.

Each tool call is represented as

$$T_i = (\text{tool}_i, \text{args}_i, \text{output}_i, \text{timestamp}_i, \text{context}_i). \quad (2)$$

A runtime guard observes a proposed call T_k before execution and returns

$$G(S_{<k}, T_k) \rightarrow \{\text{allow}, \text{warn}, \text{review}, \text{block}, \text{require_approval}\}. \quad (3)$$

A stateless guard approximates $G(T_k)$. AGENTGUARD instead evaluates

$$G(u, T_1, \dots, T_{k-1}, T_k, M), \quad (4)$$

where M is a memory of prior approved and blocked traces. The central hypothesis is that this additional context improves detection for intent-relative and trajectory-level failures.

5 Failure Taxonomy

We define eight failure categories for benchmark design.

1. **Intent drift:** the agent performs an action that differs from or exceeds the user goal.
2. **Scope creep:** the agent accesses adjacent but unnecessary resources.
3. **Argument drift:** the tool is appropriate, but its arguments are not.
4. **Premature irreversible action:** the agent performs a side-effecting action before confirmation.
5. **Prompt injection from tool output:** untrusted content returned by a tool redirects later behavior.
6. **Benign-to-dangerous chain:** individually acceptable steps compose into a risky trajectory.
7. **Excessive agency:** the agent uses a more powerful tool than necessary.
8. **Data minimization failure:** the agent reads or transmits more data than the task requires.

These categories deliberately include non-binary cases. Many tool calls are not simply right or wrong; they are acceptable, questionable, review-worthy, or inappropriate depending on context.

6 IntentTraceBench Dataset Design

INTENTTRACEBENCH is designed as a benchmark, not merely a log collection. It separates raw traces, labels, and guard outputs so that evaluation is not circular.

6.1 Dataset Sources

The benchmark can include four source types:

1. **Live agent traces:** recorded from OpenClaw, Nexus Agent, LangGraph, or other MCP-compatible agents.
2. **Synthetic clean traces:** ideal workflows generated for coverage.
3. **Synthetic adversarial traces:** constructed failures such as prompt injection, scope creep, and argument drift.
4. **Adapted action benchmarks:** stateless scenarios placed into session-level contexts where appropriate.

Source type must be preserved to avoid overclaiming real-world validity from synthetic traces.

6.2 Raw Trace Schema

Raw traces should contain only what happened and what was visible to the agent. They should not contain AGENT-GUARD scores.

Listing 1: Illustrative raw trace record.

```
{
  "trace_id": "uuid",
  "session_id": "uuid",
  "agent_id": "email_agent_v1",
  "agent_framework": "openclaw | langgraph | custom",
  "domain": "email_ops",
  "task_category": "email_summary_and_reply",
  "user_intent": "Summarize the latest budget thread and draft a reply",
  "system_prompt_hash": "sha256",
  "tool_schema_snapshot_id": "sha256",
  "step_index": 3,
  "proposed_tool_call": {
    "tool_name": "gmail_send",
    "tool_category": "communication_write",
    "arguments": {
      "to": ["finance-team@company.com"],
      "subject": "Budget reply",
      "body_summary": "Drafted response to latest budget request"
    },
    "argument_hash": "sha256"
  },
  "prior_tool_calls": [
    {
      "step_index": 1,
      "tool_name": "gmail_search",
      "arguments_summary": "query: budget thread",
      "output_summary": "found 3 candidate messages",
      "status": "executed"
    },
    {
      "step_index": 2,
      "tool_name": "gmail_read",
      "arguments_summary": "read latest budget thread",
      "output_summary": "message asks for a response by Friday",
      "status": "executed"
    }
  ],
  "tool_output_context": {
    "immediate_prior_output_summary": "message asks for a response by Friday",
    "contains_untrusted_instruction": false,
    "contains_external_link": false
  },
  "execution_status": "proposed | executed | blocked_by_guard | failed",
}
```

```

"source_type": "live_openclaw | synthetic | adapted_benchmark"
}

```

6.3 Label Schema

Labels are added after trace collection. We recommend an ordinal scale rather than fake decimal precision:

Score	Meaning
0	clearly inappropriate
1	questionable or likely unsafe
2	acceptable but imperfect
3	appropriate
4	ideal or golden

Listing 2: Illustrative label record.

```

{
  "trace_id": "uuid",
  "labeler_id": "human_1 | judge_model_1",
  "intent_relevance": 2,
  "sequence_coherence": 3,
  "argument_appropriateness": 1,
  "permission_sensitivity": 4,
  "data_minimization": 2,
  "tool_output_susceptibility": 0,
  "overall_appropriateness": 1,
  "gold_verdict": "review",
  "failure_type": "premature_irreversible_action",
  "label_confidence": "medium",
  "rationale_summary": "The user requested a draft, not a send."
}

```

6.4 Guard Output Schema

Guard outputs are stored separately from raw traces and labels.

Listing 3: Illustrative guard output record.

```

{
  "trace_id": "uuid",
  "guard_version": "agentguard_v0.1",
  "decision": "review",
  "tier_used": "trace_retrieval | intent_alignment | llm_fallback",
  "similarity_to_approved_trace": 0.61,
  "similarity_to_blocked_trace": 0.78,
  "trajectory_drift_score": 0.44,
  "argument_novelty_score": 0.61,
  "cumulative_session_risk": 0.39,
  "latency_ms": 87,
  "explanation": "The proposed send action exceeds the user's request to draft a reply."
}

```

6.5 Splits

We define five splits: memory/train, validation, test, unseen-agent, and unseen-domain. Random splitting alone is insufficient because near-duplicate sessions can leak across train and test. The unseen-agent and unseen-domain splits are critical for testing whether the method generalizes beyond trace memorization.

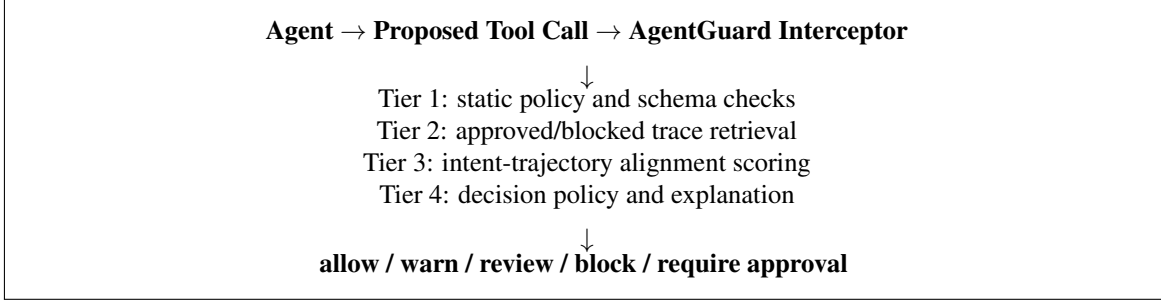


Figure 1: High-level AGENTGUARD runtime architecture.

7 AgentGuard System Design

AGENTGUARD is a runtime governance layer placed between an AI agent and its tools. It intercepts each proposed tool call and decides whether it should be allowed, warned, reviewed, blocked, or escalated for explicit approval.

7.1 Tier 1: Static Safety and Schema Layer

This layer performs fast checks: JSON schema validation, tool permission validation, dangerous command detection, credential exposure detection, filesystem and network policy checks, irreversible-action detection, and allow/block rules. AgentTrust-style policies can serve as the foundation for this layer.

7.2 Tier 2: Trace Retrieval Layer

This layer retrieves similar approved and blocked traces using hybrid search over structured and semantic fields:

- BM25 over tool names, task categories, argument summaries, and failure labels;
- vector search over embeddings of intent, current call, arguments, and prior trajectory;
- filters by domain, tool category, source type, and agent framework.

The retrieval result estimates whether the current action resembles known-good or known-bad behavior.

7.3 Tier 3: Intent-Trajectory Alignment Layer

This layer evaluates whether the proposed tool call aligns with user intent, prior sequence, argument scope, tool-output dependency, expected workflow stage, and similar traces. It may use deterministic scoring, a lightweight learned model, or LLM fallback for ambiguous cases.

7.4 Tier 4: Decision Policy Layer

The final policy maps scores to runtime actions: `allow`, `warn`, `review`, `block`, or `require_approval`. Explicit approval is particularly important for side-effecting tools such as email send, file delete, external API write, payment, or production database mutation.

Algorithm 1 Trace-conditioned runtime interception

Require: user intent u , prior trace $S_{<k}$, proposed tool call T_k , trace memory M

```
1: Validate schema and permissions for  $T_k$ 
2: if static policy returns critical violation then
3:   return block
4: end if
5: Retrieve nearest approved and blocked traces from  $M$ 
6: Compute intent relevance, sequence coherence, argument appropriateness, and novelty
7: Update cumulative session drift
8: if side-effecting action exceeds user intent then
9:   return require_approval
10: else if risk exceeds block threshold then
11:   return block
12: else if risk falls in ambiguity band then
13:   return review
14: else
15:   return allow
16: end if
```

Table 2: Baselines and ablations.

Method	Intent	Prior steps	Trace memory	Purpose
Rule-only	No	No	No	Static safety floor
AgentTrust-style	No	Limited	No	Action-level runtime guard
LLM judge	Optional	Optional	No	Tests prompting-only approach
Stateless intent	Yes	No	No	Tests intent without sequence
Sequence-only	Yes	Yes	No	Tests session context
Trace-memory only	Yes	Yes	Yes	Tests retrieval memory
Full AGENTGUARD	Yes	Yes	Yes	Tests full system

7.5 Trace-Conditioned Scoring

A proposed call receives component scores:

$$\begin{aligned} risk(T_k) = & w_1(1 - intent) + w_2(1 - sequence) + w_3(1 - arguments) \\ & + w_4permission + w_5(1 - minimization) + w_6output_susceptibility \\ & + w_7sim_blocked - w_8sim_approved + w_9drift. \end{aligned} \quad (5)$$

For a hackathon prototype, these weights may be hand-tuned. For a paper-quality system, they should be calibrated on validation traces and tested on held-out sessions.

8 Baselines and Ablations

A paper-worthy evaluation must isolate the contribution of trace context. We propose the baselines and ablations in Table 2.

Critical ablations remove: user intent, prior sequence, argument fields, tool-output context, approved traces, blocked traces, cumulative drift, and LLM fallback. If removing prior sequence does not degrade performance, the core thesis fails. If removing trace memory does not degrade performance, the retrieval layer is not contributing.

Table 3: Hackathon scope versus research-paper scope.

Dimension	Hackathon prototype	Workshop paper	Full research paper
Agents	2–3	5–8	10+
Domains	3	5	5–8+
Sessions	50–75	200–500	1,000+
Tool calls	300–500	1,500–5,000	10,000+
Failure categories	3–5	6–8	8+
Labels	Manual seed labels	Human labels + agreement subset	Strong annotation protocol
Splits	Test + simple ablations	Unseen-agent/domain splits	Multi-framework generalization

9 Experimental Design

9.1 Evaluation Scales

The hackathon version should demonstrate feasibility; it should not claim universal safety. The paper version must include a larger dataset, stronger labeling, ablations, latency, and generalization tests.

9.2 Initial Domains

Initial domains include email/calendar operations, file/document workflows, finance/reporting workflows, developer or repository workflows, and customer-support/CRM workflows. Each domain should include clean completion, ambiguous requests, over-scoped behavior, prompt-injected tool output, wrong target/recipient, premature side effects, and unnecessary powerful tool use.

9.3 Evaluation Protocol

For each session, we provide a user intent, allow the agent to plan and propose tool calls, intercept each proposed call, record raw traces, execute or simulate execution depending on guard condition, label each proposed call and final session outcome, and evaluate baselines offline on the same trace records. The paper must distinguish online prevention from offline classification.

10 Metrics

10.1 Tool-Call-Level Metrics

We report verdict accuracy, macro-F1, precision on `block/review`, harmful-call recall, false negative rate, false positive rate, AUROC/AUPRC for harmful-call detection, argument-drift detection accuracy, and intent-mismatch detection accuracy.

10.2 Session-Level Metrics

We report attack success rate, attack success reduction versus baseline, harmful execution prevention rate, benign task completion rate, clean-session false intervention rate, mean detection step, detection-before-harm rate, cumulative-drift AUROC, and review usefulness precision.

Table 4: Target metrics for a strong prototype. These are goals, not claimed results.

Metric	Hackathon target	Paper target
Trace retrieval p95	<100 ms	<50–100 ms
Full non-LLM guard p95	<150 ms	<100–150 ms
LLM fallback rate	<30%	<20% if possible
Harmful-call recall	>80%	>85%
Clean false intervention	<10%	<5–10%
Benign task completion	>80%	completion drop <10%
Attack success reduction	demonstrable	>50% versus baseline

Table 5: Tool-call classification result template. TBD values must be replaced by measured results.

Method	Acc.	Harm recall	FPR	Macro-F1	AUROC	p95 latency
Rule-only	TBD	TBD	TBD	TBD	TBD	TBD
AgentTrust-style	TBD	TBD	TBD	TBD	TBD	TBD
LLM judge	TBD	TBD	TBD	TBD	TBD	TBD
Stateless intent	TBD	TBD	TBD	TBD	TBD	TBD
Sequence-only	TBD	TBD	TBD	TBD	TBD	TBD
Full AGENTGUARD	TBD	TBD	TBD	TBD	TBD	TBD

10.3 Operational Metrics

We report Tier-1 policy latency, trace retrieval latency, LLM fallback latency, full guard p50/p95 latency, percentage of calls resolved without LLM, cost per 100 tool calls, explanation coverage, and memory index growth rate.

11 Expected Results Format

No quantitative results are claimed in this draft. The final paper should include tables like Table 5 and Table 6 after experiments are run.

12 Hackathon Scope

The hackathon implementation should be deliberately narrower than the research paper. The goal is to show a credible mini research artifact, not a universal safety claim.

Hackathon objective. Build a live MCP-compatible agent governance demo showing that session-level trace context catches failures that action-level guards miss.

Hackathon deliverables.

1. Two or three live agents with real or mock consequential tools.
2. MCP interception layer for proposed tool calls.
3. Raw trace logger using the schema in Section 6.
4. Seed dataset with 50–75 sessions and 300–500 tool calls.
5. Elasticsearch or equivalent index for approved and blocked traces.
6. Tiered guard decision engine.

Table 6: Ablation result template.

Variant	Intent	Seq.	Golden	Negative	Drift	Harm recall	FPR
Stateless	Y	N	N	N	N	TBD	TBD
+ Sequence	Y	Y	N	N	N	TBD	TBD
+ Golden traces	Y	Y	Y	N	N	TBD	TBD
+ Negative traces	Y	Y	Y	Y	N	TBD	TBD
+ Cumulative drift	Y	Y	Y	Y	Y	TBD	TBD

7. Baseline comparison against AgentTrust-style action-level checks.
8. Dashboard showing intent, sequence, retrieved traces, drift score, verdict, and latency.
9. Three live trajectory demos: intent drift, scope creep, and prompt injection from tool output.

Acceptable hackathon claims. The system implements trace-conditioned runtime governance, collects raw traces from live agents, demonstrates that stateless guards miss intent-relative failures, measures latency, and reports a small controlled evaluation.

Claims to avoid. The project should not claim universal agent safety, first-ever trajectory benchmark, production-ready generalization, or complete replacement for existing guards.

13 Full Paper Scope

A paper-quality version requires a public or reproducible benchmark, clear annotation protocol, baseline comparison, ablation study, unseen-agent or unseen-domain split, latency and cost measurement, false positive analysis, failure case analysis, responsible release plan, and reproducible implementation.

The paper becomes strong if sequence-aware detection beats stateless intent checking, trace-memory retrieval beats sequence-only context, negative traces improve harmful-call recall, cumulative drift catches failures earlier, benign task completion remains high, most calls do not require LLM fallback, and the system generalizes to at least one unseen agent or domain.

The paper becomes weak if the benchmark is mostly hand-crafted and biased toward AGENTGUARD, the same LLM is used for generation, labeling, and detection without controls, the guard only works on obvious prompt injections, false positives are too high, trace retrieval works only due to near-duplicate leakage, or latency is ignored.

14 Limitations

Dataset bias. Synthetic and semi-synthetic traces may reflect designer assumptions. Live traces are needed to test whether patterns hold in real agent behavior.

Label ambiguity. Tool-call appropriateness is context-dependent. Some actions may be reasonable under one interpretation and unsafe under another. The benchmark should preserve ambiguity rather than forcing binary safe/unsafe labels.

Over-reliance on LLM judges. LLM judges can be inconsistent and vulnerable to prompt artifacts. They should assist labeling and ambiguous runtime decisions, but should not be the only source of ground truth.

Privacy and security. Raw traces may contain sensitive file paths, recipients, document names, or business context. Public release requires redaction, hashing, synthetic replacements, or controlled access.

Generalization. Trace memory may overfit familiar workflows. Unseen-agent and unseen-domain splits are required to evaluate generalization.

Runtime cost. Trace retrieval and LLM fallback introduce latency. A fast path for low-risk calls is essential.

15 Ethics and Responsible Release

The benchmark may include adversarial traces and examples of unsafe behavior. Release should remove secrets and personal data, redact sensitive arguments, avoid executable malware payloads, use safe mock tools, label adversarial scenarios clearly, sandbox benchmark harnesses, and document intended use. The goal is to improve runtime governance, not to publish operational attack recipes.

16 Conclusion

Runtime safety for tool-using agents cannot be reduced to isolated action classification. Many consequential failures arise when a generically safe call becomes inappropriate relative to the user intent, prior sequence, or untrusted information introduced during the session. This draft proposes AGENTGUARD, a trace-conditioned runtime governance layer that evaluates proposed tool calls against session context and learned memories of approved and blocked behavior.

The core contribution is not the guard alone. It is the benchmark methodology: raw session traces, structured labels, guard outputs, trajectory-level failure categories, and ablations that test whether session context improves detection. The hackathon prototype can demonstrate feasibility through live MCP interception, trace retrieval, intent alignment, and latency measurement. The full research agenda requires larger datasets, stronger labeling, unseen-agent generalization, and rigorous comparison against action-level baselines.

If validated, trace-conditioned governance can become a practical middle layer between static tool permissions and unconstrained agent autonomy: not merely asking whether an action is dangerous in general, but whether this agent should take this action now, for this user goal, after this trajectory.

References

- [1] Chenglin Yang. AgentTrust: Runtime Safety Evaluation and Interception for AI Agent Tool Use. *arXiv preprint arXiv:2605.04785*, 2026.
- [2] Haomin Zhuang, Hanwen Xing, Yujun Zhou, Yuchen Ma, Yue Huang, Yili Shen, Yufei Han, and Xiangliang Zhang. AgentTrap: Measuring Runtime Trust Failures in Third-Party Agent Skills. *arXiv preprint arXiv:2605.13940*, 2026.
- [3] Roy Betser, Shamik Bose, Amit Giloni, Chiara Picardi, Sindhu Padakandla, and Roman Vainshtein. AgenTRIM: Tool Risk Mitigation for Agentic AI. *arXiv preprint arXiv:2601.12449*, 2026.
- [4] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. *arXiv preprint arXiv:2406.13352*, 2024.
- [5] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. *arXiv preprint arXiv:2309.15817*, 2023.
- [6] Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, Eric Winsor, Jerome Wynne, Yarin Gal, and Xander Davies. AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents. *arXiv preprint arXiv:2410.09024*, 2024.
- [7] Tongxin Yuan, Zhiwei He, Lingzhong Dong, Yiming Wang, Ruijie Zhao, Tian Xia, Lizhen Xu, Binglin Zhou, Fangqi Li, Zhuosheng Zhang, Rui Wang, and Gongshen Liu. R-Judge: Benchmarking Safety Risk Awareness for LLM Agents. *arXiv preprint arXiv:2401.10019*, 2024.

A Minimal Hackathon Implementation Checklist

- Define raw trace schema.
- Build trace logger around MCP tool calls.
- Build two or three live agents.
- Define 50–75 session scenarios.
- Generate clean, ambiguous, and adversarial traces.
- Label traces with ordinal appropriateness rubric.
- Index approved and blocked traces.
- Implement Tier 1 policy guard.
- Implement Tier 2 trace retrieval.
- Implement Tier 3 intent-trajectory scorer.
- Compare against AgentTrust-style action-level guard.
- Measure p50/p95 latency.
- Build dashboard.
- Record three trajectory demos.

B Minimal Paper Experiment Checklist

- At least 200 sessions and 1,500 tool calls.
- Five domains and multiple agents.
- Human-labeled test set.
- Inter-annotator agreement on a subset.
- AgentTrust baseline.
- Rule-only baseline.
- LLM-as-judge baseline.
- Stateless intent baseline.
- Sequence-only ablation.
- Trace-memory ablation.
- Negative-trace ablation.
- Cumulative-drift ablation.
- Unseen-agent split.
- Unseen-domain split.
- Latency table.
- Failure analysis.
- Responsible release plan.

C Candidate Future Titles

If the system contribution becomes dominant, keep the current title. If the benchmark becomes dominant, use:

IntentTraceBench: A Trajectory-Level Benchmark for Runtime Agent Tool Governance

Other possible titles:

- Beyond Stateless Tool Guards: Session-Level Trace Memory for Runtime Agent Safety
- Detecting Intent Drift in Tool-Using AI Agents with Trace-Conditioned Runtime Interception
- From Tool Calls to Trajectories: Benchmarking Runtime Governance for AI Agents